

GreenWaste

Gestion intelligente des déchets industriels

Présenté par: Chamekh Raghed A03
Ladjimi Mariem A07
Jani Sima A07
Chebbi Maha A04
Zaghib Bechir A04

Livrable L1 — Product Backlog & Sprint Planning:

Membre	Rôle Scrum	Fonctionnalité
Raghed	Product Owner + Dev	Authentification
Mariem	Scrum Master + Dev	Planification collectes
Sima	Dev	Alertes & conformité
Maha	Dev	Tableau de bord admin
Bechir	Dev	Facturation

1. Fiche Projet

1.1 Présentation

GreenWaste est une plateforme de gestion intelligente des déchets industriels en Tunisie. Elle permet aux usines de déclarer, suivre et optimiser leur collecte de déchets en temps réel. L'objectif est de faciliter la conformité réglementaire et de réduire l'impact environnemental des industries tunisiennes.

Problème résolu :

Les industries tunisiennes génèrent des volumes importants de déchets (chimiques, solides, recyclables) souvent mal tracés, mal triés et non conformes aux normes environnementales. Il n'existe pas de système centralisé pour coordonner producteurs, collecteurs et inspecteurs.

Cible (utilisateurs visés) :

- Responsables HSE (Hygiène, Sécurité, Environnement) des usines
- Opérateurs industriels chargés de la saisie des données
- Sociétés de collecte et transporteurs de déchets
- Inspecteurs environnementaux
- Administrateurs de la plateforme

1.2 Acteurs du système

Acteur	Rôle	Actions principales
Opérateur industriel	Utilisateur de base	Déclare les déchets produits dans son usine
Responsable HSE	Superviseur	Valide les déclarations, planifie les collectes, reçoit les alertes
Collecteur	Prestataire	Confirme les collectes, génère les factures
Inspecteur	Régulateur	Consulte la conformité de chaque usine
Administrateur	Admin plateforme	Gère les comptes, surveille le tableau de bord global

1.3 Fonctionnalités et répartition

Membre	Rôle Scrum	Fonctionnalité	Branche Git
Raghd	Product Owner + Dev	Authentification & gestion des comptes	feature/authentification
Mariem	Scrum Master + Dev	Planification des collectes	feature/planification-collecte
Sima	Dev	Alertes & conformité réglementaire	feature/alertes-conformite
Maha	Dev	Tableau de bord admin & rapports	feature/admin-dashboard
Bechir	Dev	Facturation entre usine et collecteur	feature/facturation

2. Product Backlog

User Stories: En tant que [rôle], je veux [action] afin de [bénéfice].

Chaque membre est responsable de 2 User Stories. Total vélocité : 58 Story Points.

US-01	Inscription utilisateur	En tant qu'opérateur industriel, je veux créer un compte afin d'accéder à la plateforme.	Must	1. Formulaire avec nom, email, mot de passe, nom usine. 2. Email de confirmation envoyé. 3. Erreur si email existant. 4. Mot de passe 8 caractères minimum.	5	Raghd
US-02	Connexion / Déconnexion	En tant qu'utilisateur enregistré, je veux me connecter avec email/mot de passe afin d'accéder à mon espace sécurisé.	Must	1. Connexion réussie avec identifiants valides. 2. Erreur après 3 tentatives échouées. 3. Redirection vers tableau de bord. 4. Déconnexion efface la session.	3	Raghd
US-03	Planifier une collecte	En tant que responsable HSE, je veux planifier une collecte afin d'organiser l'enlèvement des déchets dans les délais réglementaires.	Must	1. Formulaire avec date, type déchet, collecteur. 2. Notification envoyée au collecteur. 3. Collecte visible dans le calendrier. 4. Impossible de planifier à une date passée.	8	Mariam
US-04	Suivre le statut d'une collecte	En tant que responsable HSE, je veux suivre le statut de mes collectes afin de savoir si elles sont confirmées ou effectuées.	Should	1. Statut : En attente / Confirmée / Effectuée / Annulée. 2. Mise à jour automatique quand le collecteur confirme. 3. Historique consultable sur 12 mois.	5	Mariam
US-05	Alerte dépassement de seuil	En tant que responsable HSE, je veux recevoir une alerte automatique quand le volume dépasse le seuil réglementaire afin d'agir avant toute sanction.	Must	1. Alerte à 80% et 100% du seuil. 2. Indique type déchet, volume actuel, seuil autorisé. 3. Visible dans tableau de bord et envoyée par email. 4. Inspecteur notifié à 100%.	8	Sim
US-06	Conformité des usines	En tant qu'inspecteur, je veux consulter l'état de conformité de chaque usine afin de prioriser mes contrôles sur le terrain.	Should	1. Indicateur : Conforme / À risque / Non conforme. 2. Détail volumes déclarés vs seuils. 3. Filtre par statut de conformité.	5	Sim
US-07	Tableau de bord admin	En tant qu'administrateur, je veux visualiser	Must	1. Affiche : usines inscrites, collectes du jour, alertes actives,	8	Maha

		un tableau de bord global afin de surveiller l'activité de toutes les usines et collecteurs.		volume total. 2. Filtres par région et type de déchet. 3. Rafraîchissement auto toutes les 5 min. 4. Graphique évolution mensuelle.		
US-08	Gestion des comptes	En tant qu'administrateur, je veux gérer les comptes utilisateurs afin de valider les inscriptions et modifier les rôles.	Should	1. Liste complète des utilisateurs avec rôle et statut. 2. Activation, suspension ou suppression de compte. 3. Email envoyé à l'utilisateur lors de toute modification.	5	Maha
US-09	Générer une facture	En tant que collecteur, je veux générer automatiquement une facture après chaque collecte afin de facturer l'usine en dinars tunisiens.	Must	1. Facture générée dès collecte marquée Effectuée. 2. Contient date, type déchet, quantité kg, prix unitaire DT, total TTC. 3. Téléchargeable et envoyée par email. 4. Numéro unique attribué.	8	Bechir
US-10	Historique des factures	En tant que responsable HSE, je veux consulter l'historique de toutes les factures afin de suivre les dépenses liées à la gestion des déchets.	Could	1. Liste avec date, collecteur, montant DT, statut payée/en attente. 2. Filtres par mois et type de déchet. 3. Total des dépenses du mois affiché.	3	Bechir

Espaces

GW

Résumé
Chronologie
Backlog
Tableau
Calendrier
Liste
Formulaires
Objectifs
Développement
Code
More

Recherchez dans...
ML BZ ES MC RC
Filtrer

Backlog
(10 tickets)
58 0 0
Créer un sprint

☒	GW-1 US1 : Inscription utilisateur	À FAIRE	5	RC
☒	GW-2 US2 : Connexion / Déconnexion	À FAIRE	3	RC
☒	GW-3 US3: Planifier une collecte	À FAIRE	8	ML
☒	GW-4 US4: Suivre le statut d'une collecte	À FAIRE	5	ML
☒	GW-5 US5: Alerte dépassement de seuil	À FAIRE	8	ES
☒	GW-6 US6: Conformité des usines	À FAIRE	5	ES
☒	GW-7 US7: Tableau de bord admin	À FAIRE	8	MC
☒	GW-8 US8: Gestion des comptes	À FAIRE	5	MC
☒	GW-9 US9: Générer une facture	À FAIRE	8	BZ
☒	GW-10 US10: Historique des factures	À FAIRE	3	BZ

+ Créer

3. Sprint 1 Planning

3.1 Sprint Goal

Concevoir l'expérience utilisateur (maquettes Figma) et produire l'architecture Java exécutable du système GreenWaste.

Critère de succès mesurable : Le Main.java compile et s'exécute, affichant les objets de base de chaque fonctionnalité. Les maquettes Figma sont navigables et validées par le Product Owner.

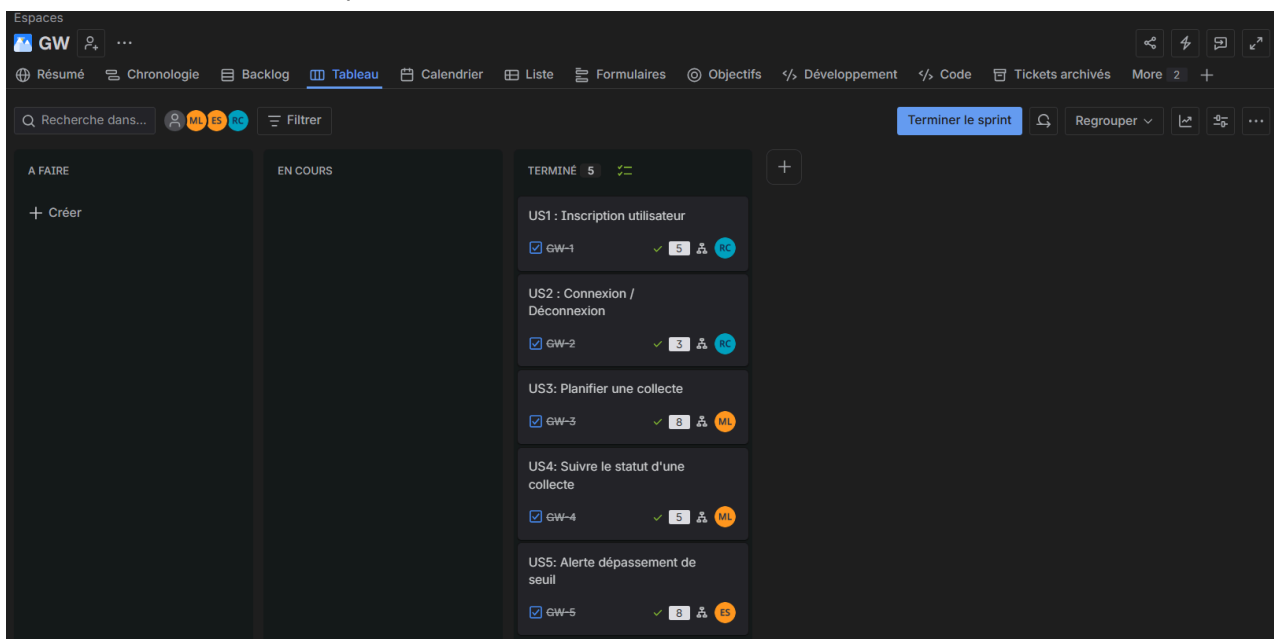
3.2 User Stories sélectionnées (Must)

- US-01 — Inscription utilisateur (Raghd) — 5 SP
- US-02 — Connexion / Déconnexion (Raghd) — 3 SP
- US-03 — Planifier une collecte (Mariem) — 8 SP
- US-05 — Alerte dépassement de seuil (Sima) — 8 SP
- US-07 — Tableau de bord admin (Maha) — 8 SP
- US-09 — Générer une facture (Bechir) — 8 SP

Total Sprint 1 : 40 Story Points

3.3 Definition of Done — Sprint 1

- Le code compile sans erreur
- Le Main.java exécute le scénario de base de chaque fonctionnalité
- La classe est intégrée sur main via Pull Request
- Au moins un collègue a relu le code (review sur GitHub)
- Les maquettes Figma sont navigables (minimum 2 écrans par membre)
- Le PO a validé les maquettes



● 3. Sprint 2 Planning

3.1 Sprint Goal

Livrer un prototype complet avec la logique métier réelle et des scénarios de bout en bout fonctionnels dans le Main.java V2.

Critère de succès mesurable : Le Main.java V2 exécute des scénarios complets où les classes interagissent entre elles, une collecte déclenche une alerte, une facture est générée automatiquement et l'administrateur voit les statistiques en temps réel.

3.2 User Stories sélectionnées

US reportées du Sprint 1 et nouvelles US :

- US-02 — Connexion/Déconnexion (Raghd) — 3 SP
- US-04 — Suivre le statut d'une collecte (Mariem) — 5 SP
- US-06 — Conformité des usines (Sim) — 5 SP
- US-08 — Gestion des comptes (Maha) — 5 SP
- US-10 — Historique des factures (Bechir) — 3 SP

Total Sprint 2 : 21 Story Points

3.3 Ajustement de vélocité

	Sprint 1	Sprint 2
Vélocité prévue	40 SP	21 SP
Vélocité réalisée	40 SP	En cours
Ajustement	—	Focus sur logique métier réelle

3.4 Definition of Done — Sprint 2

- Les méthodes métier contiennent la vraie logique et pas seulement des `System.out.println`
- Les classes de différents membres interagissent entre elles
- Le Main.java V2 exécute minimum 3 scénarios complets de bout en bout
- Les prompts IA sont documentés avec le code généré et le code final corrigé
- Le code est intégré via Pull Request avec review d'un collègue
- Les montants sont en Dinar Tunisien
- Le rapport final PDF est rédigé et consolidé

Espaces

GW

...

Résumé

Chronologie

Backlog

Tableau

Calendrier

Liste

Formulaires

Objectifs

Développement

Code

Tickets archivés

More

2

+

Recherche dans...

ML

BZ

ES

MC

Filtrer

Terminer le sprint

Regrouper

...

A FAIRE

+ Créer

EN COURS

TERMINÉ 5

...

+

US6: Conformité des usines

GW-6

✓

5

ES

US7: Tableau de bord admin

GW-7

✓

8

MC

US8: Gestion des comptes

GW-8

✓

5

MC

US9: Générer une facture

GW-9

✓

8

BZ

US10: Historique des factures

GW-10

✓

3

BZ

4. Modélisation UML

DCU :

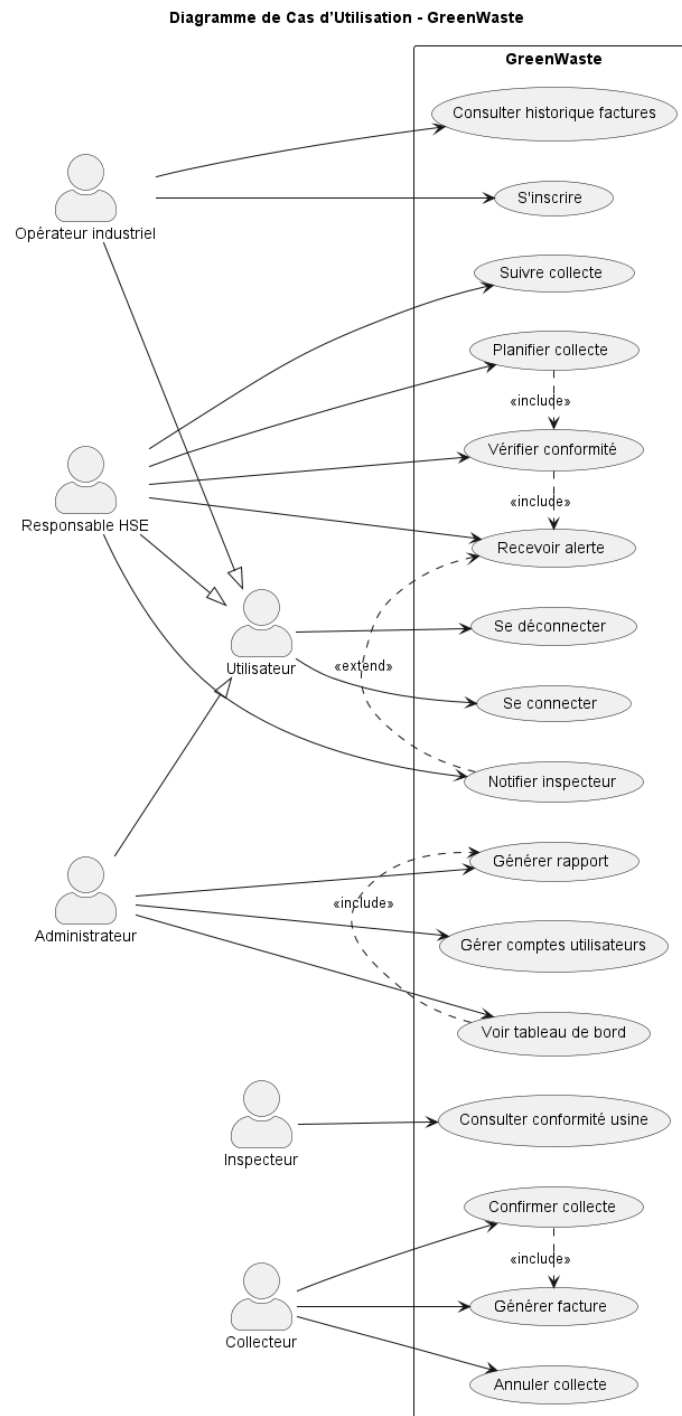


Diagramme de cas d'utilisation: Le diagramme de cas d'utilisation de **GreenWaste** illustre les fonctionnalités principales offertes par le système ainsi que les acteurs qui les utilisent. L'acteur générique **Utilisateur** regroupe les actions communes comme se connecter et se déconnecter, tandis que des rôles spécifiques en héritent, tels que l'Opérateur industriel, le Responsable HSE et l'Administrateur. L'opérateur industriel peut s'inscrire et consulter l'historique des factures. Le Responsable HSE assure la planification des collectes, le suivi des opérations, la vérification de conformité et la réception des alertes. L'Administrateur gère les comptes utilisateurs, consulte le

tableau de bord et génère des rapports. L'Inspecteur contrôle la conformité des usines, alors que le Collecteur confirme ou annule les collectes et génère les factures. Les relations <<include>> et <<extend>> permettent de représenter les dépendances et extensions entre certains cas d'utilisation.

DC :

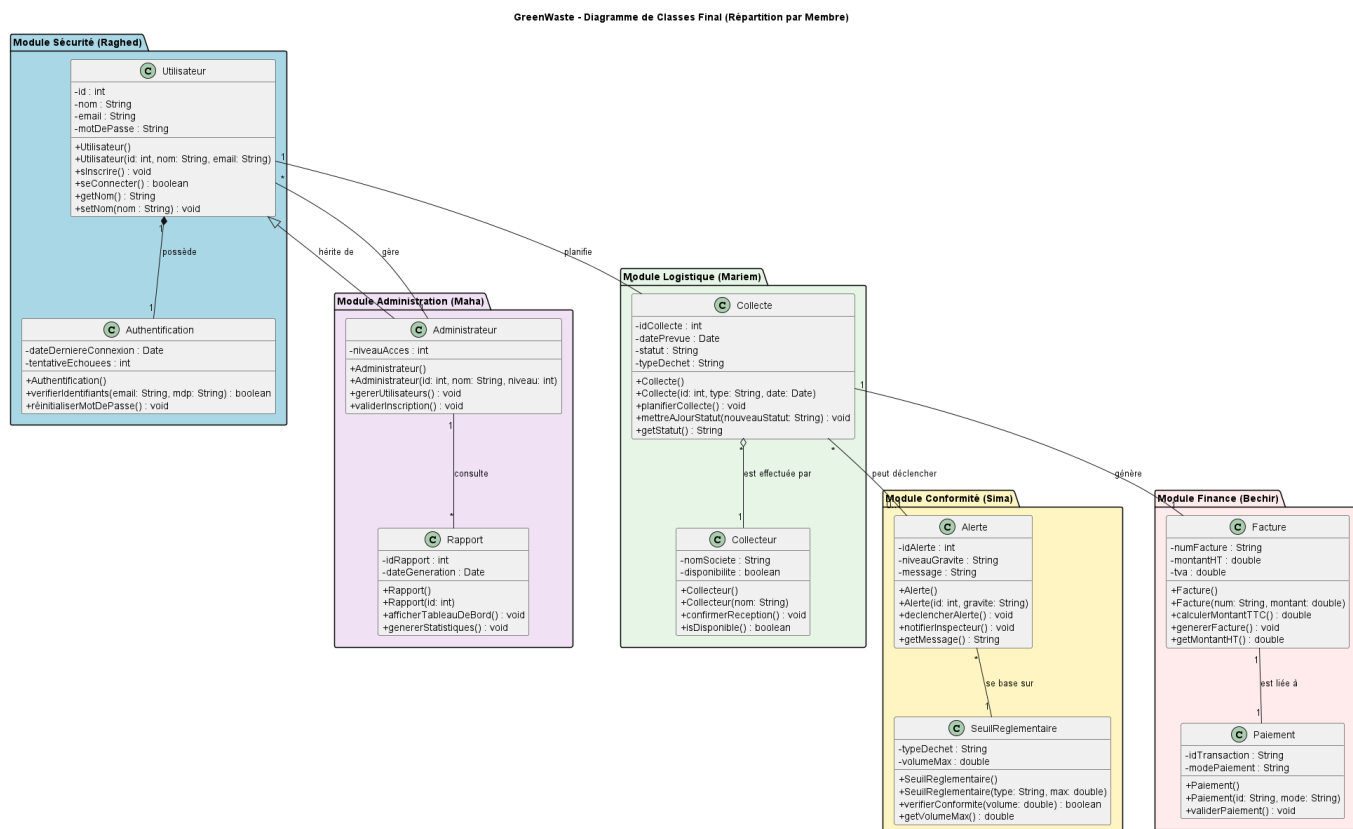
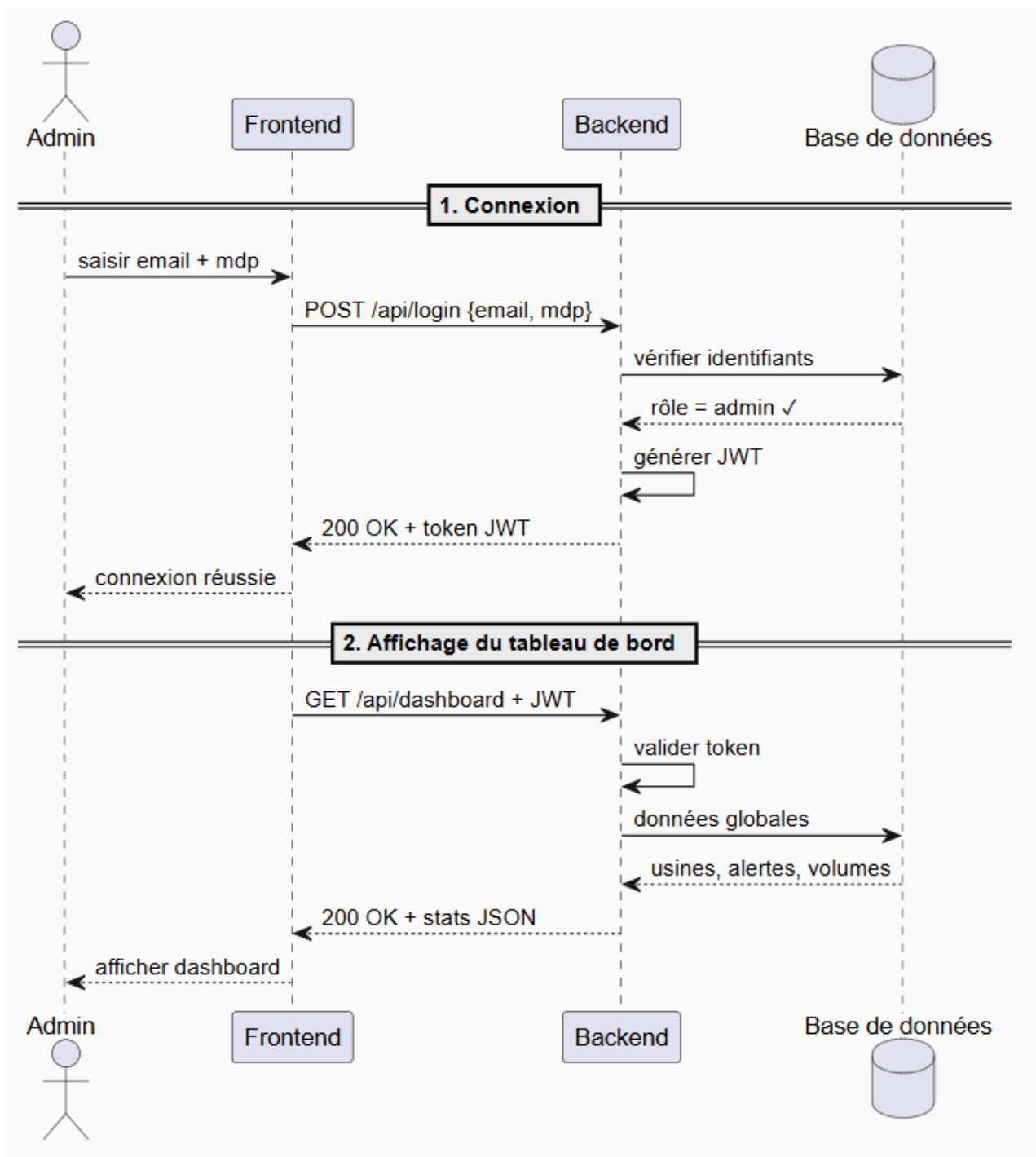


Diagramme de classes: Le diagramme de classes de **GreenWaste** modélise les principales composantes de l'application à travers cinq modules complémentaires. Le module sécurité gère les utilisateurs et leur authentification. Le module administration permet à l'administrateur de gérer les comptes et consulter les rapports. Le module logistique s'occupe de la planification et du suivi des collectes via les classes **Collecte** et **Collecteur**. Le module conformité comprend les classes **Alerte** et **SeuilReglementaire**, utilisées pour contrôler les seuils autorisés et déclencher des alertes en cas de dépassement. Enfin, le module finance assure la génération des factures et la gestion des paiements. Ce diagramme montre une architecture claire et structurée basée sur les relations entre classes.



DS :

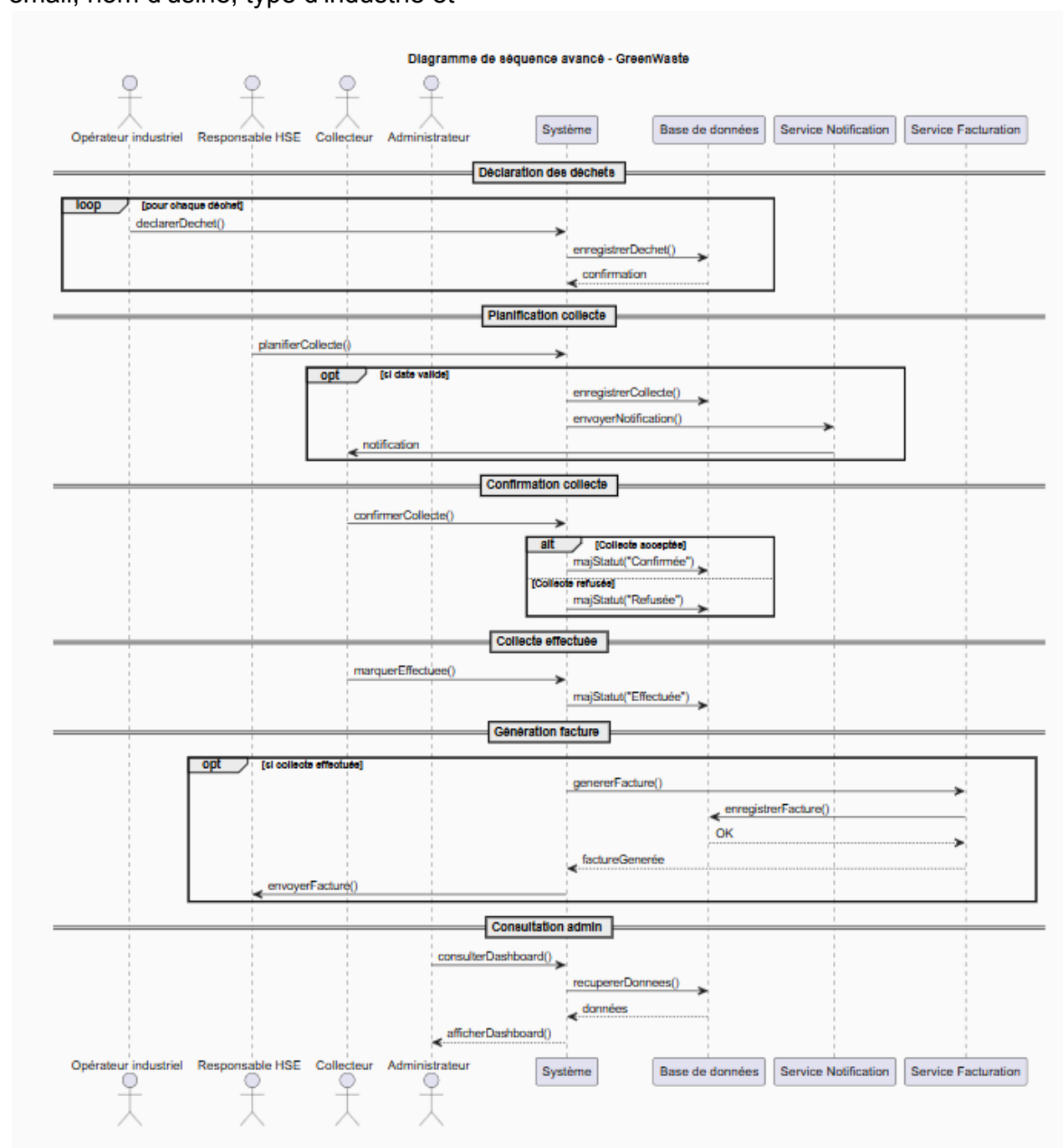
Diagramme de Séquence sprint 1: Ce diagramme illustre deux étapes principales. D'abord la **connexion** : l'administrateur saisit son email et mot de passe, le Backend vérifie ses identifiants dans la Base de données, confirme son rôle admin et génère un token JWT renvoyé au Frontend. Ensuite l'**affichage du tableau de bord** : le Frontend envoie une requête avec le token JWT, le Backend la valide, récupère les données globales (usines, alertes, volumes) depuis la Base de données et les retourne au Frontend qui affiche le dashboard à l'administrateur.

Diagramme de séquence complexe : Ce diagramme de séquence représente le fonctionnement global du système GreenWaste, depuis la déclaration des déchets jusqu'à la facturation et la consultation par l'administrateur. Il montre les interactions entre les acteurs et le système, ainsi que les différentes étapes comme la planification, la confirmation et l'exécution de la collecte. Des conditions sont utilisées pour gérer certains cas, comme la validation de la date ou l'acceptation de la collecte, ce qui permet d'assurer un déroulement logique et cohérent du processus.

5. Maquettes Figma :

Parcours 1 — Inscription et Connexion

L'utilisateur ouvre l'application et arrive sur l'écran de connexion. S'il n'a pas encore de compte il clique sur "Créer un compte" et remplit le formulaire avec son prénom, nom, email, nom d'usine, type d'industrie et



mot de passe. Après avoir cliqué sur "Créer mon compte" il est redirigé vers l'écran de connexion où il entre son email et mot de passe pour accéder à l'application.

 INSCRIPTION

Rejoignez GreenWaste

R remplissez les informations ci-dessous pour créer votre profil industriel et accéder à la plateforme.

Nom

 Dupont

Prénom

 Jean

Email professionnel

 jean.dupont@usine.fr

Mot de passe



Minimum 8 caractères

DÉTAILS INDUSTRIELS

Nom de l'usine

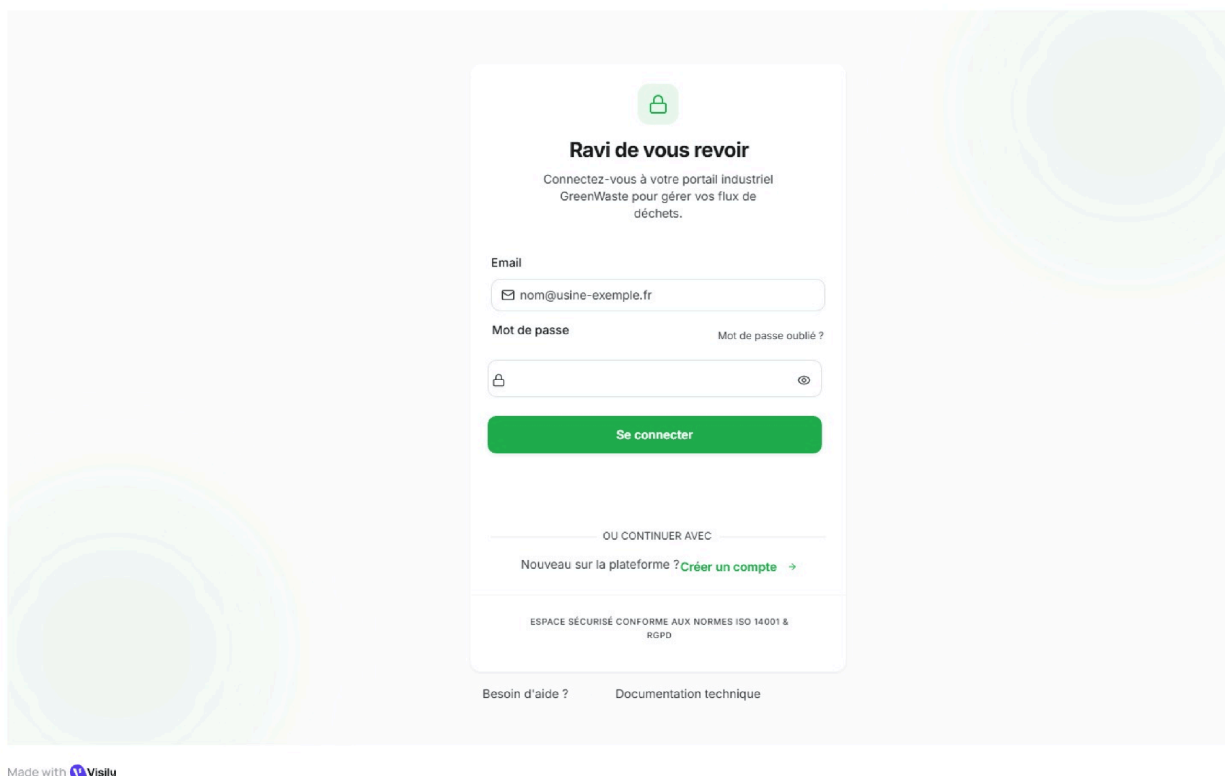
 EcoValor Usine Sud

Type d'industrie

 Centre de Recyclage

Créer le compte >

Déjà un compte ? [Se connecter](#)



Parcours 2 — Planifier une collecte

Une fois connecté l'utilisateur clique sur le bouton "+" dans l'écran liste des collectes ou sur l'onglet "Planifier" dans la barre de navigation. Il remplit le formulaire en choisissant la date de passage, le type de déchet et le collecteur assigné. Il clique sur "Planifier la collecte" et la collecte apparaît dans sa liste avec le statut "En attente".



Planifier une collecte

Type de déchet

Déchets Organiques

Date de la collecte



21 avril 2026

Statut initial

Planifié

Nom du collecteur

Michel Bernard



Planifier



Annuler

Made with Visily



MENU PRINCIPAL

Tableau de bord

Liste des collectes

Planning

Types de déchets

ADMINISTRATION

Paramètres

Déconnexion



Jean Dupont
Responsable Logistique

Tableau de bord > Collectes

Rechercher une collecte...



+ Nouvelle Collecte

Gestion des Collectes

Gérez et suivez le déploiement de vos équipes sur le terrain en temps réel.

Total Collectes

1,284



En attente

12



Complétées

842



Incidents

3



Liste des collectes de déchets

Aperçu détaillé des tournées de ramassage par secteur.

Exporter (CSV)

Imprimer

ID	Type de déchet	Localisation	Date & Heure	Statut	Actions
COL-8291	Carton et Papiers	Zone Industrielle Nord	20 Avr 2026 09:00	Planifiée	Annuler
COL-8292	Verre Cullet	Centre Ville - Dépôt A	19 Avr 2026 14:30	Effectuée	Annuler
COL-8293	Déchets Organiques	Résidence les Pins	19 Avr 2026 11:15	Annulée	
COL-8294	Métaux et Ferraille	Quai de la Gare	18 Avr 2026 08:45	Effectuée	Annuler
COL-8295	Plastiques (PET/PEHD)	Supermarché Horizon	21 Avr 2026 10:00	Planifiée	Annuler

Affichage de 5 sur 124 collectes

Précédent

1

2

3

...

Suivant

Made with Visily

Parcours 3—Contrôle de conformité et gestion des alertes

Une fois la collecte planifiée, le système exécute automatiquement une vérification via le module Seuils et Alertes. La quantité saisie est comparée aux limites légales pour déterminer le niveau de risque :

Vigilance (80% du seuil) : Le système affiche un avertissement visuel pour prévenir d'une saturation imminente.

Alerte Critique (100% et plus) : Le système bloque la validation normale, génère une alerte prioritaire et active le protocole de notification vers l'inspecteur.

GreenAlerts

Alertes Actives

Surveillance en temps réel des capacités de stockage et conformité réglementaire.

Analyses

Paramètres

Déconnexion

Rechercher une alerte...

SYSTÈME DE SURVEILLANCE

Admin Greenwaste
Gestionnaire de site

Mise à jour : Aujourd'hui, 14:42

Total des Alertes
12
+2 nouveaux depuis hier

Alertes Critiques
02

Inspections Requises
03

Volume Global (kg)
3,905
92% de la capacité depuis hier

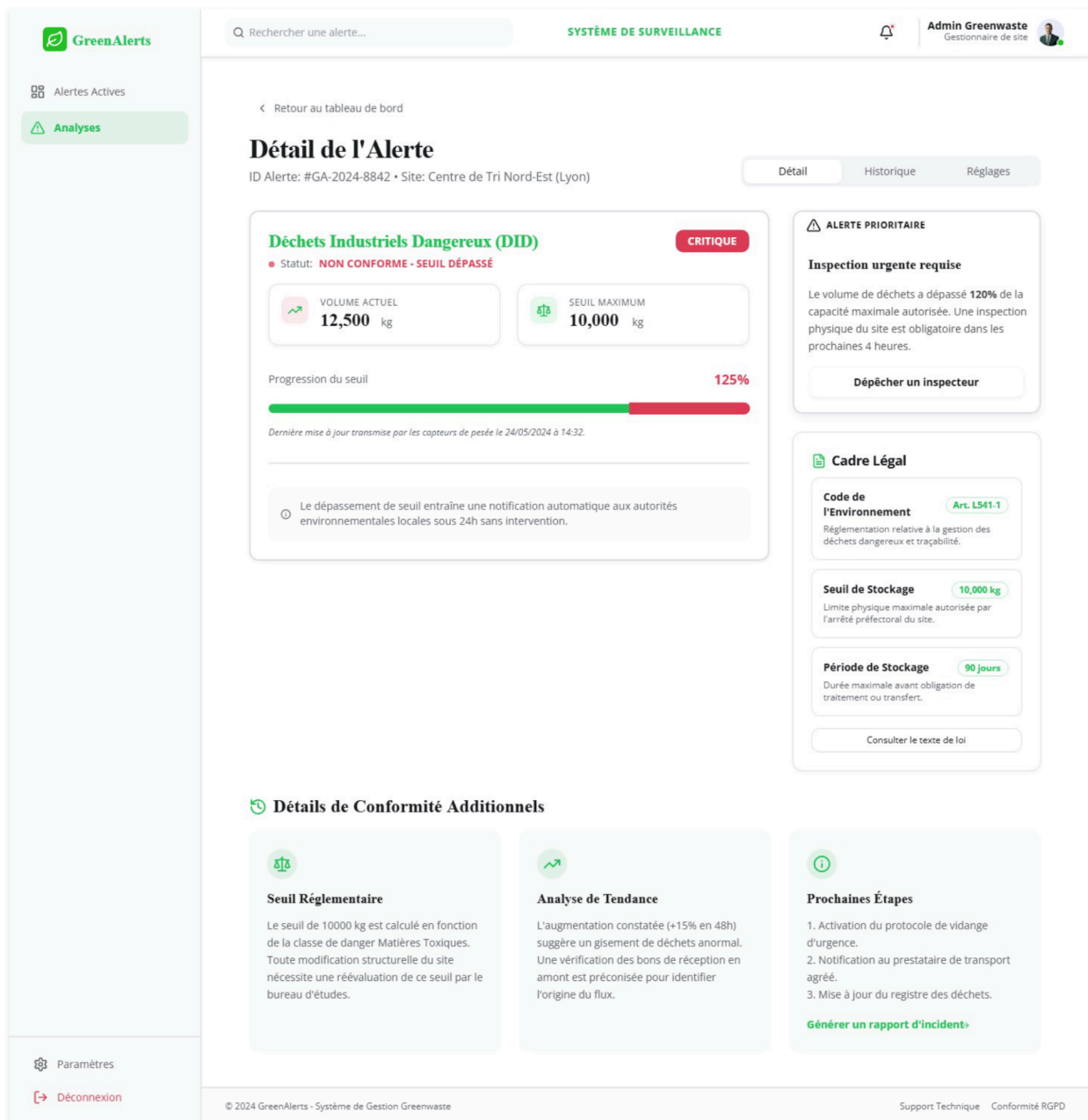
Liste des flux de déchets

TousUrgentsConformes

Type de déchet	Volume actuel (kg)	Seuil max (kg)	Statut
Déchets Chimiques Solvants	1,450	1,200	Alerte déclenchée - Seuil dépassé
Plastiques Industriels (PET)	890	1,000	Inspection urgente requise
Déchets Organiques Biodégradables	450	1,500	Conforme
Métaux Lourds (Plomb/Cadmium)	125	100	Alerte déclenchée - Seuil dépassé
Cartons et Emballages	780	800	Inspection urgente requise
Verre de Laboratoire	210	500	Conforme

© 2024 GreenAlerts - Système de Gestion Greenwaste

Support TechniqueConformité RGPD



Made with Visily

Parcours 4 — Facturation et suivi des paiements

Dès qu'une collecte est confirmée comme effectuée, le module de facturation s'active automatiquement. La classe `Facture.java` génère une facture avec un numéro unique, calcule le montant TTC avec la TVA à 19% et l'envoie par email à l'usine concernée.

Du côté de la Gestion des Factures, le responsable HSE ou le collecteur dispose d'une vue complète sur toutes les factures émises, filtrables par type de déchet (Plastique, Organique, Carton, Verre). Chaque ligne affiche le client, la quantité collectée en tonnes, le montant en TND et le statut de paiement (Payé / En attente / En retard), permettant un suivi précis et rapide.

Du côté de la Gestion des Paiements, l'administrateur suit en temps réel les encaissements du mois avec trois indicateurs clés : les recettes totales, les montants en attente et les impayés en retard nécessitant une relance. Chaque transaction est enregistrée avec sa méthode de règlement (Virement Bancaire, Chèque ou Espèces). Un rapport mensuel complet est générable en PDF directement depuis l'interface pour faciliter la comptabilité de la plateforme.

Factures

Paiements

Paramètres

Déconnexion

Interface GreenWaste

Admin GreenWaste
Tunis, TN

Gestion des Paiements

Suivi des encaissements et historique des transactions financières.

Filtres Avancés

Nouveau Paiement

Recettes Totales (Mai)

45,820.000 DT

+12.4% vs mois dernier

En Attente de Règlement

8,430.500 DT

-2.1% 8 factures actives

Impayés / En Retard

3,120.000 DT

+0.5% Nécessite relance

Tous

Payés

En attente

En retard

Rechercher un client, une ID...

ID Transaction	Date	Client / Entreprise	Montant	Méthode	Statut	Actions
PAY-2026-001	14 Mai 2026	EcoIndustries Tunis	4,250.000 DT	Virement	Payé	Download Arrow
PAY-2026-002	12 Mai 2026	Sousse Recyclage	1,890.500 DT	Chèque	En attente	Valider Download Arrow
PAY-2026-003	10 Mai 2026	Municipauté de Carthage	12,400.000 DT	Virement	Payé	Download Arrow
PAY-2026-004	08 Mai 2026	Hôtel Azur Hammamet	3,120.000 DT	Espèces	En retard	Valider Download Arrow
PAY-2026-005	05 Mai 2026	Pharmacie Centrale	950.000 DT	Chèque	Payé	Download Arrow

Affichage de 5 sur 128 transactions

Précédent Suivant

Récapitulatif des Méthodes

Répartition des paiements reçus ce mois-ci.

Virement Bancaire

32,100 DT

Chèque

10,600 DT

Espèces

3,120 DT

Rapport Mensuel

Générez et téléchargez le rapport complet des encaissements de Mai 2026 pour la comptabilité.

Télécharger PDF

© 2026 Interface GreenWaste — Administration Financière Tunisie

Interface GreenWaste

Admin GreenWaste
Tunis, TN

Factures

Paielements

Paramètres

Déconnexion

Gestion des Factures

Suivez et générez vos factures de collecte de déchets en temps réel.

Nouvelle Facture

Total Facturé (Mars)
42,850 TND
+12% vs mois dernier

Poids Total Collecté
124.8 T
+5.4% vs mois dernier

Factures Impayées
8

Rechercher un client

Filtres avancés

ToutPlastiqueOrganiqueCarton

N° Facture	Client	Déchet	Quantité (T)	Montant (TND)	Statut
FAC-2026-001	Ecoindustrie Sousse 12 Mars 2026	Plastique PET	12.5	5 625,000	Payé
FAC-2026-002	Sodexo Tunis 14 Mars 2026	Organique	8.2	1 476,000	En attente
FAC-2026-003	Bizerte Logistique 15 Mars 2026	Carton	25.0	5 500,000	Payé
FAC-2026-004	Clinique Pasteur 16 Mars 2026	Verre	4.8	1 488,000	En retard
FAC-2026-005	Hôtel Sahara Beach 18 Mars 2026	Plastique PET	15.3	6 885,000	En attente

Affichage de 5 sur 142 factures

PrécédentSuivant

Détails Client Actif

Informations de facturation courantes

ENTREPRISE
Ecoindustrie Sousse SA
Zone Industrielle Sidi Abdelhamid, 4000 Sousse

CONTACT
Ahmed Mansour
ahmed.mansour@ecoindustrie.tn

Total 5,625,000
Période : TND

Quantité Totale : 12.5 T

Exporter le Relevé

© 2026 Interface GreenWaste — Administration Financière Tunisie

Made with Visily

Parcours 5— Tableau de bord administrateur et gestion des comptes

Une fois connecté, l'administrateur accède directement au tableau de bord global qui affiche en temps réel les indicateurs clés de la plateforme : le nombre d'usines inscrites, les collectes du jour, les alertes actives et le volume total de déchets traités. Il peut affiner sa vue en appliquant des filtres par région ou par type de déchet, et les données se rafraîchissent automatiquement toutes les 5 minutes. En naviguant vers l'onglet "Comptes" dans la barre de navigation, il accède à la liste complète des utilisateurs avec leur rôle et leur statut. Il peut alors activer un compte en attente, suspendre un utilisateur ou modifier son rôle en cliquant sur le bouton d'actions correspondant, et l'utilisateur concerné reçoit automatiquement un email de notification suite à toute modification.

NAVIGATION PRINCIPALE

Dashboard

Gestion Utilisateurs

Statistiques

Paramètres

● Système Opérationnel

Toutes les unités de traitement fonctionnent normalement. Dernière mise à jour il y a 2 min.

Tableau de Bord Admin

Surveillance globale des activités et flux de déchets.

Exporter PDF

Actualiser



LIVE

Usines inscrites

12

+2 ce mois-ci



LIVE

Collectes du jour

05

3 terminées, 2 en cours



LIVE

Alertes actives

03

Urgent: Capacité bac 80%



LIVE

Volume total

48 200 kg

85% de la capacité totale

Filtrer par :



Rafraîchissement : toutes les 5 min

Rechercher...

Gestion des Utilisateurs

Liste du personnel administratif et opérationnel actif sur le terrain.



ID	Nom & Email	Rôle	Statut	Actions
#001	Ali Ben Salem ali.bs@greenwaste.tn	Opérateur	Actif	Modifier Désactiver
#002	Sonia Triki sonia.ti@greenwaste.tn	Responsable HSE	Actif	Modifier Désactiver
#003	Marc Lefebvre m.lefebvre@greenwaste.tn	Analyste Data	Inactif	Modifier Désactiver
#004	Imen Mansour imen.m@greenwaste.tn	Superviseur	Actif	Modifier Désactiver
#005	Karim Dridi k.dridi@greenwaste.tn	Opérateur	En attente	Modifier Désactiver

Affichage de 5 sur 28 utilisateurs enregistrés

Précédent

Suivant

Régions les plus actives

Tunis Centre

Collecte hebdomadaire

12,400 kg

+12% croissant

Ariana

Collecte hebdomadaire

8,200 kg

+5% croissant

Sfax Sud

Collecte hebdomadaire

15,600 kg

+18% croissant

Prochaines Collectes

AOÛT

14

Zone Industrielle Ben Arous

Déchets Plastiques & Métaux • Camion #42

08:30 AM

Priorité Haute

AOÛT

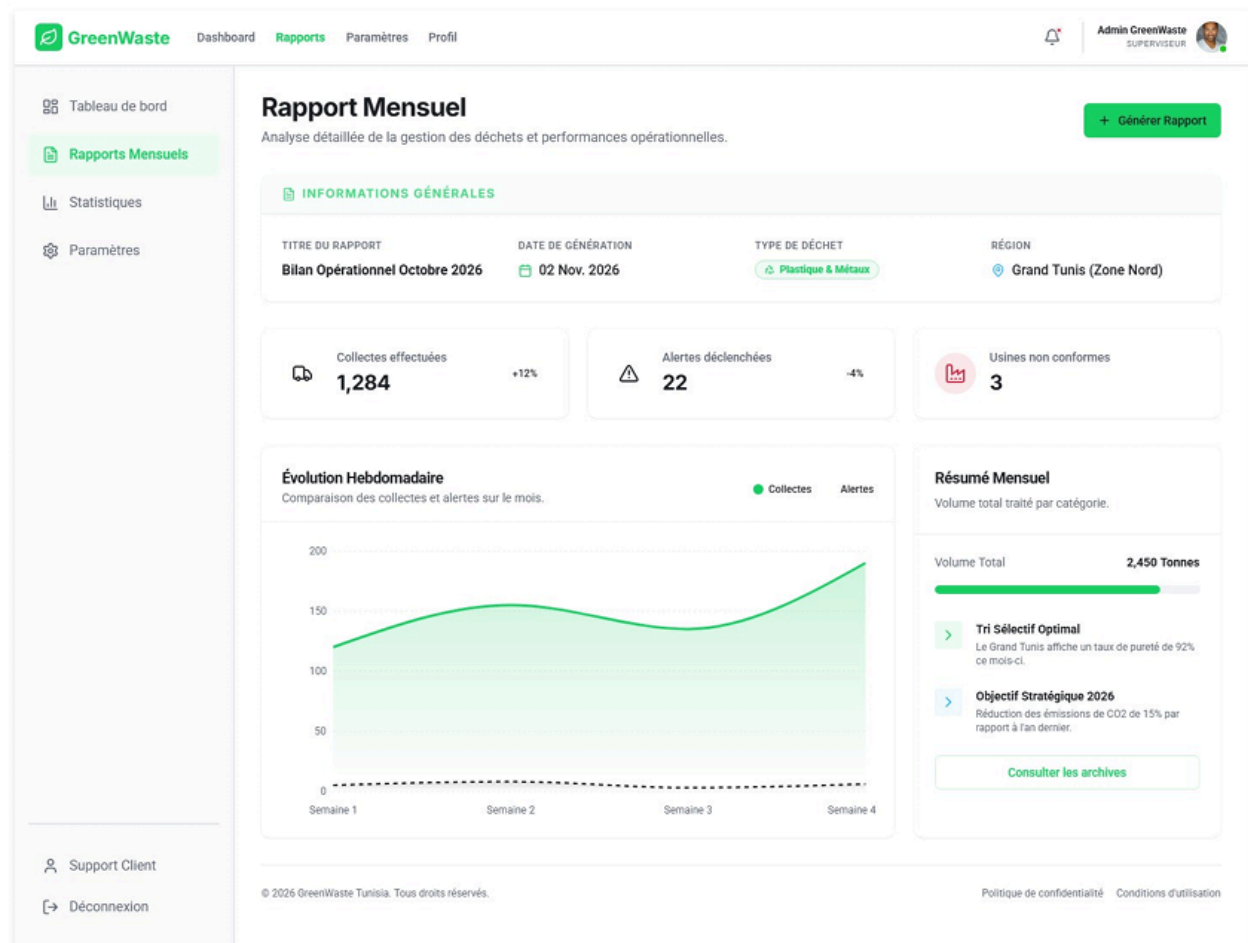
15

Quartier Lac 2

Papier & Carton • Camion #18

10:00 AM

Voir tout le calendrier →



lienprototype:figma:<https://www.figma.com/make/QpuTYPVsclwdUuRJl7DdBp/GreenWaste-management?t=zQOgibokil8mqmQx-1>

6. Code —Approche hybride

X Authentification (Raghd) :

1. Prompt :

une méthode Java inscrire(id, nom, email, motDePasse, role) pour une classe Authentification qui gère une List<Utilisateur>.

Elle doit vérifier que l'email n'est pas déjà utilisé, que le mot de passe fait min 6 caractères, créer l'utilisateur, hasher son mot de passe et l'ajouter à la liste.

2. Code généré par IA :

```
public boolean inscrire(int id, String nom,
    String email, String mdp, String role) {
    Utilisateur u = new Utilisateur(id, nom, email, role);
    utilisateursInscrits.add(u);
    return true;
}
```

3. Code corrigé :

```
public boolean inscrire(int id, String nom,String email, String motDePasse,String role) {
    if (nom == null || nom.trim().isEmpty()) {
        System.out.println("Nom obligatoire !");
        return false;
    }
    if (email == null || email.indexOf("@") < 1|| !email.contains(".")) {
        System.out.println("Email invalide !");
        return false;
    }
    for (Utilisateur u : utilisateursInscrits) {
        if (u.getEmail().equalsIgnoreCase(email)) {
            System.out.println( "Email deja utilise : " + email);
            return false;
        }
    }
    if (motDePasse == null || motDePasse.length() < 6) {
        System.out.println( "Mot de passe min 6 caracteres !");
        return false;
    }
    Utilisateur nouveau = new Utilisateur( id, nom.trim(), email.toLowerCase(), role);
    nouveau.setMotDePasseHash(nouveau.hasherMotDePasse(motDePasse));
    nouveau.setCompteActif(true);
    utilisateursInscrits.add(nouveau);
    System.out.println("Compte cree : "+ nom + " (" + role + ")");
    return true;
}
```

X Collecte (Mariem) :

1. Prompt :

Générer une méthode Java pour planifier une collecte de déchets avec validation de la date et mise à jour du statut.

2. Code généré par IA :

```
public void planifierCollecte() {
    if (date != null && !date.isEmpty()) {
        this.statut = "Planifiée";
        System.out.println("Collecte planifiée pour le " + date);
    } else {
        System.out.println("Date invalide");
    }
}
```

3. Code corrigé :

```
public class Collecte {  
    private String typeDechet;  
    private String date;  
    private String statut;  
  
    public Collecte(String typeDechet, String date, String statut) {  
        this.typeDechet = typeDechet;  
        this.date = date;  
        this.statut = statut;  
    }  
  
    public void planifierCollecte() {  
        if (date == null || date.trim().isEmpty()) {  
            System.out.println("Erreur : date invalide");  
            return;  
        }  
  
        if (statut != null && statut.equalsIgnoreCase("Annulée")) {  
            System.out.println("Impossible de planifier une collecte annulée");  
            return;  
        }  
  
        if (statut != null && statut.equalsIgnoreCase("Planifiée")) {  
            System.out.println("Collecte déjà planifiée");  
            return;  
        }  
  
        this.statut = "Planifiée";  
        System.out.println("Collecte planifiée avec succès pour le " + date);  
    }  
  
    public void annulerCollecte() {  
        if (statut != null && statut.equalsIgnoreCase("Confirmée")) {
```

```

        System.out.println("Impossible d'annuler une collecte déjà confirmée");
        return;
    }

    this.statut = "Annulée";
    System.out.println("Collecte annulée");
}

public String getStatut() {
    return statut;
}

public void setStatut(String statut) {
    this.statut = statut;
}

public String getDate() {
    return date;
}
}

```

X Alertes (Sima) :

1. Prompt :

Générer une méthode Java `verifierEtAlerter()` qui vérifie si une quantité dépasse le seuil réglementaire et retourne une Alerte CRITIQUE ou ATTENTION.

2. Code généré par IA :

```

public Alerte verifierEtAlerter(double quantiteSaisie) {
    String statut;

    if (quantiteSaisie >= this.seuilMax) {
        statut = "CRITIQUE";

        Alerte nouvelleAlerte = new Alerte(this.typeDechet, quantiteSaisie, statut);
        nouvelleAlerte.notifierInspecteur();
    }
}

```



```

        return nouvelleAlerte;
    }
    else if (quantiteSaisie >= this.seuilMax * 0.8) {
        statut = "ATTENTION";
        System.out.println("[INFO] Seuil d'alerte proche pour : " + typeDechet);
        return new Alerte(this.typeDechet, quantiteSaisie, statut);
    }

    return null;
}

```

3. Code corrigé :

```

public class SeuilReglementaire {

    private String typeDechet;
    private double seuilMax;

    public SeuilReglementaire() {}

    public SeuilReglementaire(String typeDechet, double seuilMax) {
        this.typeDechet = typeDechet;
        this.seuilMax = seuilMax;
    }

    public Alerte verifierEtAlerter(double quantiteSaisie) {
        String statut;

        if (quantiteSaisie >= this.seuilMax) {
            statut = "CRITIQUE";

            Alerte nouvelleAlerte = new Alerte(this.typeDechet, quantiteSaisie, statut);
            nouvelleAlerte.notifierInspecteur();
            return nouvelleAlerte;
        }
        else if (quantiteSaisie >= this.seuilMax * 0.8) {
            statut = "ATTENTION";
            System.out.println("[INFO] Seuil d'alerte proche pour : " + typeDechet);
            return new Alerte(this.typeDechet, quantiteSaisie, statut);
        }

        return null;
    }
}

```

```

    public double getSeuilMax() { return seuilMax; }
    public void setSeuilMax(double seuilMax) { this.seuilMax = seuilMax; }
}

```

X Admin (Maha) :

1. Prompt :

Méthode Java afficherStatistiquesFacturation(List<Facture>) avec Streams :
calculer total, TTC, PAYEE, EN_ATTENTE, moyenne, taux (%).

Utiliser printf, éviter division par 0.

Retourner la méthode propre.

2. Code généré par IA :

```

public void afficherStatistiquesFacturation(List<Facture> factures) {
    double total = factures.stream().mapToDouble(Facture::getMontantTTC).sum();
    System.out.println("Total: " + total);
}

```

3. Code corrigé :

```

public void afficherStatistiquesFacturation(List<Facture> factures) {

```

```

    if (factures == null || factures.isEmpty()) {
        System.out.println("\n Aucune donnée de facturation.\n");
        return;
    }

```

```

    long nbFactures = factures.size();

```

```

    double totalTTC = factures.stream()
        .mapToDouble(Facture::getMontantTTC)
        .sum();

```

```

    double totalPayees = factures.stream()
        .filter(f -> "PAYEE".equals(f.getStatut()))
        .mapToDouble(Facture::getMontantTTC)
        .sum();

```

```

    double totalEnAttente = factures.stream()
        .filter(f -> "EN_ATTENTE".equals(f.getStatut()))
        .mapToDouble(Facture::getMontantTTC)
        .sum();

```

```

    double moyenne = totalTTC / nbFactures;

```

```
double tauxRecouvrement = totalTTC > 0
    ? (totalPayees / totalTTC) * 100
    : 0;
```

```
System.out.println("\n===== DASHBOARD ADMIN — FACTURATION
=====");
```

```
System.out.printf ("|| Nombre de factures   : %-27d || %n", nbFactures);
System.out.printf ("|| Total TTC           : %-27.3f || %n", totalTTC);
System.out.printf ("|| Total payé          : %-27.3f || %n", totalPayees);
System.out.printf ("|| En attente          : %-27.3f || %n", totalEnAttente);
System.out.printf ("|| Moyenne / facture    : %-27.3f || %n", moyenne);
System.out.printf ("|| Taux recouvrement   : %-26.1f%% %n", tauxRecouvrement);
```

```
// KPI décisionnel admin
```

```
if (tauxRecouvrement < 50) {
    System.out.println(" Alerte: Faible recouvrement !");
} else if (tauxRecouvrement < 80) {
    System.out.println(" Surveillance recommandée.");
} else {
    System.out.println(" Situation financière saine.");
}
}
```

X Facturation (Bechir) :

Classe Facture.java

1. Prompt

Cree une classe Java Facture avec les champs : id, numeroFacture (format FAC-2026-id), dateEmission, montantDT, statut (EN_ATTENTE par default), destinataire. Ajoute getters, setStatut() et toString().

2. Code IA

```
public class Facture {
    private int id;
    private String numeroFacture;
    private double montantDT;
    private String statut;

    public Facture(int id, double montant) {
        this.id = id;
        this.montantDT = montant;
        this.statut = "EN_ATTENTE";
    }
    public String toString() { return id + " | " + montantDT; }
}
```

3. Code corrigé

```
package com.greenwastemodels;

public class Facture {
    private int id;
    private String numeroFacture;
    private String dateEmission;
    private double montantDT;
    private String statut;
    private String destinataire;

    public Facture(int id, String destinataire,
        double montantDT, String dateEmission) {
        this.id = id;
        this.numeroFacture = "FAC-2026-" + id;
        this.destinataire = destinataire;
        this.montantDT = montantDT;
        this.dateEmission = dateEmission;
        this.statut = "EN_ATTENTE";
    }
    public String getNumeroFacture() { return numeroFacture; }
    public double getMontantDT() { return montantDT; }
    public String getStatut() { return statut; }
    public String getDestinataire() { return destinataire; }
    public void setStatut(String s){ this.statut = s; }

    @Override
    public String toString() {
        return "Facture[" + numeroFacture + " ] "
            + destinataire + " | "
            + String.format("%.3f", montantDT)
            + " DT | " + statut;
    }
}
```

1. genererFacture()

1. Prompt *Ecris une methode Java genererFacture(id, destinataire, montant, date) pour une classe Paiement dans GreenWaste. Verifier que le destinataire n est pas vide, que le montant est positif, creer une Facture et l ajouter a historiqueFactures.*

2. Code IA

```
public Facture genererFacture(int id,
    String dest, double montant, String date) {
    return new Facture(id, dest, montant, date);
}
```

3. Code corrige

```
public Facture genererFacture(int id, String destinataire,
    double montant, String date) {
    if (destinataire == null || destinataire.trim().isEmpty()) {
        System.out.println(" Destinataire obligatoire !");
        return null;
    }
    if (montant <= 0) {
        System.out.println(" Montant invalide !");
        return null;
    }
    Facture f = new Facture(id, destinataire, montant, date);
    historiqueFactures.add(f);
    System.out.printf(" Facture generee : %s | %s | %.3f DT%n",
        f.getNumeroFacture(), destinataire, montant);
    return f;
}
```

2. effectuerPaiement()

1. Prompt

Methode Java effectuerPaiement(montant, methode, facture) pour une classe Paiement GreenWaste. Verifier montant positif, methode parmi VIREMENT/ESPECES/CHEQUE, montant suffisant par rapport a la facture. Montants en DT.

2. Code IA

```
public boolean effectuerPaiement(double montant,
    String methode, Facture f) {
    if (montant <= 0) return false;
    this.statut = "VALIDE";
    f.setStatut("PAYEE");
    return true;
}
```

3. Code corrige

```
public boolean effectuerPaiement(double montant,
    String methode, Facture facture) {
    if (montant <= 0) {
        System.out.println("Montant invalide !"); return false; }
    if (methode == null || (!methode.equals("VIREMENT")
        && !methode.equals("ESPECES")
        && !methode.equals("CHEQUE")))) {
        System.out.println(" Methode invalide ! (VIREMENT/ESPECES/CHEQUE)");
        return false; }
    if (facture == null) {
        System.out.println(" Facture introuvable !"); return false; }
    if (montant < facture.getMontantDT()) {
        System.out.printf(" Montant insuffisant ! Requis: %.3f DT%n",
            facture.getMontantDT()); return false; }
    this.montantDT = montant; this.methodePaiement = methode;
    this.statut = "VALIDE"; facture.setStatut("PAYEE");
    System.out.printf(" Paiement : %.3f DT via %s | Facture %s PAYEE%n",
        montant, methode, facture.getNumeroFacture());
    return true;
}
```

3. annulerPaiement()

1. Prompt: *Methode annulerPaiement(facture) dans Paiement qui verifie que statut est VALIDE, remet le statut a ANNULE et la facture a EN_ATTENTE.*

2. Code IA:

```
public boolean annulerPaiement() {  
    this.statut = "ANNULE";  
    return true;  
}
```

3. Code corrigé:

```
public boolean annulerPaiement(Facture facture) {  
    if (!this.statut.equals("VALIDE")) {  
        System.out.println(" Impossible d annuler !"); return false; }  
    if (facture == null) {  
        System.out.println(" Facture introuvable !"); return false; }  
    this.statut = "ANNULE";  
    facture.setStatut("EN_ATTENTE");  
    System.out.println(" Paiement annule | Facture "  
        + facture.getNumeroFacture() + " remise EN_ATTENTE");  
    return true;  
}
```

4. afficherHistorique()

1. Prompt: *Methode afficherHistorique() qui parcourt historiqueFactures et affiche chaque facture avec le total des montants payes en DT.*

2. Code IA:

```
public void afficherHistorique() {  
    for (Facture f : historiqueFactures)  
        System.out.println(f);  
}
```

3. Code corrige:

```
public void afficherHistorique() {  
    if (historiqueFactures.isEmpty()) {  
        System.out.println("Aucune facture !"); return; }  
    System.out.println("=== Historique Factures ===");  
    double total = 0;  
    for (Facture f : historiqueFactures) {  
        System.out.println(" " + f);  
        if (f.getStatut().equals("PAYEE"))  
            total += f.getMontantDT();  
    }  
    System.out.printf("Total paye : %.3f DT%n", total);  
}
```

7. TABLEAU COMPARATIF AGL VS IA

Critère	AGL pur (Sprint 1)	IA pure	Hybride (votre approche)
Cohérence structurelle	Garantie par le diagramme de classes	Variable selon le prompt	Structure AGL + code IA
Rapidité d'implémentation	Méthodes vides, squelette seulement	Code généré rapidement	Rapide et structuré
Reproductibilité	Déterministe	Probabiliste, résultats variables	Contrôlée par review humaine
Qualité du code métier	Pas de logique réelle	À vérifier et corriger	Vérifié et corrigé par le membre
Respect des règles métier	Défini dans les US Jira	IA ne connaît pas vos US	IA guidée par les US du backlog
Intégration entre classes	Classes isolées sans interaction	Interaction possible mais incohérente	Interaction réelle testée dans Main.java
Gestion des erreurs	Aucune validation	Basique selon le prompt	Validations ajoutées manuellement
Montants en DT	Non applicable au squelette	Peut oublier la devise	DT intégré et vérifié

8. SPRINT REVIEWS & RETROSPECTIVES

Raghd chamekh : Product Owner

Sprint 1 Review

Durant le Sprint 1 j'ai rédigé les 10 user stories du product backlog avec les critères d'acceptation, les priorités MoSCoW et les story points. J'ai créé les deux écrans Figma correspondant à ma fonctionnalité : la page de connexion et la page d'inscription. J'ai également codé les classes `Utilisateur.java` et `Authentification.java` avec les méthodes `sInscrire()`, `seConnecter()` et `seDeconnecter()`. Les US-01 et US-02 ont été livrées comme prévu.

Mariem Ladjimi : Scrum Master

Sprint 1 Rétrospective

Ce qui a bien marché : la répartition des tâches était claire, chaque membre savait ce qu'il devait faire. Les maquettes Visily ont été faites rapidement.

Ce qui n'a pas marché : la configuration de GitHub et les branches ont pris beaucoup de temps. La structure des packages Java a causé des erreurs de compilation.

Ce que nous ferons différemment : configurer GitHub dès le début du sprint avant de commencer à coder.

Raghd chamekh : Product Owner

Sprint 2 Review

J'ai complété les méthodes métier de `Authentication.java` avec la vraie logique de validation, mis à jour le Kanban sur Jira et participé à l'intégration du `Main.java` V2 avec des scénarios complets.

Mariem Ladjimi : Scrum Master

Sprint 2 Rétrospective

Ce qui a bien marché : l'approche hybride AGL et IA a permis de générer du code rapidement tout en gardant une bonne qualité grâce aux corrections manuelles.

Ce qui n'a pas marché : la coordination entre les membres pour intégrer toutes les classes dans le `Main.java` final a été difficile.

Ce que nous ferions différemment : faire des réunions plus régulières entre membres pour vérifier que les classes s'intègrent bien ensemble.

Vélocité

	Sprint 1	Sprint 2
Vélocité prévue	29 points	29 points
Vélocité réalisée	8 points	8 points

9.BILAN INDIVIDUEL

Raghd (Product Owner)

Durant ce projet GreenWaste j'ai joué le rôle de Product Owner ce qui m'a permis de comprendre l'importance de bien définir les besoins avant de commencer à coder. J'ai rédigé les user stories du backlog en m'assurant que chaque fonctionnalité avait des critères d'acceptation clairs et mesurables.

Ma contribution : j'ai créé les maquettes de connexion et d'inscription sur Visily, codé les classes `Utilisateur.java` et `Authentication.java` et participé à l'intégration dans le `Main.java`.

Ce que j'ai appris : j'ai appris à utiliser Git et GitHub pour travailler en équipe avec des branches, à utiliser Jira pour gérer un backlog Scrum et à coder en Java en suivant une architecture orientée objet.

Mes difficultés : la configuration de l'environnement Java dans VS Code a été compliquée au début, la gestion des packages et des imports causait des erreurs difficiles à comprendre. La prise en main de GitHub et des branches a également demandé du temps.

Mariem (Scrum Master)

Durant ce projet, j'ai occupé le rôle de Scrum Master, ce qui m'a permis de mieux comprendre l'organisation et la gestion d'un projet en méthodologie Agile. J'ai assuré la gestion du projet sur Jira, notamment la création et le suivi du backlog, la planification des sprints ainsi que la répartition des tâches entre les membres de l'équipe.

Ma contribution : j'ai mis en place et structuré le projet sur GitHub (création du repository, organisation des dossiers), réalisé certaines maquettes de l'application, et développé les classes *Collecte.java* et *Collecteur.java*. J'ai également participé à l'intégration globale du projet avec l'équipe.

Ce que j'ai appris : ce projet m'a permis de maîtriser l'utilisation de Jira dans un contexte Scrum (gestion des user stories, suivi des sprints), ainsi GitHub pour le travail collaboratif (branches, commits, pull requests). J'ai aussi renforcé mes compétences en programmation orientée objet en Java et en structuration d'un projet logiciel.

Mes difficultés : j'ai rencontré des difficultés principalement avec GitHub, notamment la gestion des branches et des pull requests, ainsi que la compréhension du workflow collaboratif. L'intégration du code entre les membres de l'équipe a également nécessité plusieurs ajustements. Enfin, certaines parties du code Java ont demandé du temps pour assurer une bonne cohérence entre les classes.

Bechir(dev)

Ma contribution au projet

Au cours des deux Sprints du projet Green Waste, j'ai pris en charge le module Facturation & Paiement dans sa totalité. Voici le détail de mes contributions :

Sprint 1	Conception des User Stories US-09 et US10, rédaction des critères d'acceptation, participation au Product Backlog et estimation en Story Points (8 SP + 3 SP).
Sprint 2	Développement complet de Paiement.java (4 méthodes) et Facture.java. Création de Main.java pour les tests. Réalisation des maquettes Figma (écran génération + historique). Contribution au diagramme de classes UML.

Ce que j'ai appris: Approche hybride IA + code manuel : J'ai appris à formuler des prompts précis et à évaluer critiqueusement le code généré par l'IA. Le code initial de l'IA était fonctionnel mais incomplet et les validations et la gestion d'erreurs étaient absentes.

Difficultés rencontrées

- Limites de l'IA sur la logique métier : Le code généré ignorait les règles spécifiques au contexte (méthodes de paiement autorisées, format DT). J'ai dû lire et corriger ligne par ligne plutôt que simplement copier-coller.
- Cohérence des statuts entre Paiement et Facture : Gérer le cycle de vie des statuts a nécessité plusieurs itérations avant d'obtenir un comportement correct.
- Synchronisation en équipe : Certaines classes dont dépendait mon module (ex. Authentification) n'étaient pas encore finalisées lors du Sprint 2, ce qui m'a contraint à travailler avec des données simulées.
- Gestion du temps : La rédaction du rapport en parallèle du développement a été difficile à équilibrer. J'aurais dû commencer la documentation plus tôt dans le Sprint.

Sima El Jani (Développeuse)

Ma contribution : Dans ce projet, j'ai conçu le moteur de surveillance environnementale de l'application. J'ai développée les classes `SeuilReglementaire.java` et `Alerte.java`. Mon rôle était de créer un système capable de comparer en temps réel les quantités de déchets saisies avec les limites légales. J'ai mis en place une logique à deux niveaux : une notification d'avertissement ("ATTENTION") lorsque l'on approche de la limite, et une alerte "CRITIQUE" avec notification automatique à l'inspecteur en cas de dépassement.

Ce que j'ai appris : Ce travail m'a permis de comprendre comment structurer une communication entre classes en Java : la classe `SeuilReglementaire` agit comme un contrôleur qui décide de créer, ou non, un objet de la classe `Alerte`. J'ai appris à utiliser des conditions logiques pour automatiser des décisions métier et à formater des messages de sortie console professionnels pour l'utilisateur final.

Mes difficultés : La principale difficulté a été de faire le pont entre mes classes et le reste du projet. Il a fallu s'assurer que les objets `Alerte` générés par mon module pouvaient être correctement stockés et affichés dans le `Main.java` et les rapports de mes camarades. La gestion des types de données et la synchronisation des noms de méthodes entre nous ont nécessité plusieurs phases de tests et de corrections pour éviter les erreurs de compilation.

Maha (Dev — Tableau de bord admin & rapports)

Durant ce projet GreenWaste j'ai joué le rôle de développeuse responsable du tableau de bord administrateur et de la gestion des comptes utilisateurs, ce qui m'a permis de comprendre l'importance d'une interface centralisée pour surveiller

l'ensemble des activités d'une plateforme en temps réel. J'ai rédigé les user stories US-07 et US-08 en définissant des critères d'acceptation précis comme le rafraîchissement automatique toutes les 5 minutes et l'envoi d'email lors de toute modification de compte.

Ma contribution : j'ai créé les deux maquettes du tableau de bord global et de la gestion des utilisateurs sur Visily, codé les classes `Administrateur.java` et `Rapport.java` avec les méthodes `afficherTableauDeBord()`, `gererUtilisateurs()`, `activerCompte()` et `suspendreCompte()`, réalisé le diagramme de séquence PlantUML illustrant le scénario de connexion admin, et connecté mes classes avec celles de Mariem (`Collecte.java`), Sima (`Alerte.java`) et Bechir (`Facture.java`) pour afficher les données en temps réel dans le dashboard. J'ai également rédigé le tableau comparatif AGL vs IA et participé à l'intégration finale dans le `Main.java`.

Ce que j'ai appris : j'ai appris à concevoir une interface admin complète en pensant à l'expérience utilisateur avant de coder, à utiliser les streams Java pour filtrer et calculer des statistiques dynamiquement, à travailler avec Git et GitHub en branches pour intégrer le code de plusieurs membres sans conflit, et à utiliser PlantUML pour modéliser des interactions entre classes sous forme de diagramme de séquence.

Mes difficultés : la connexion entre mon module et les classes des autres membres a été la partie la plus délicate, car elle dépendait de la disponibilité du code de Mariem, Sima et Bechir. Il fallait coordonner les noms des méthodes et les types de données échangés pour éviter les erreurs de compilation. La gestion du temps entre le développement de la logique métier, la rédaction du rapport et la préparation de la soutenance a également été un défi en fin de Sprint 2.